

Tokenization and Embedding

Dr. Adnan Arefeen

North South University

What is Tokenization?

Definition: Tokenization is the process of converting raw text into smaller units (tokens) that a language model can understand.

Example:

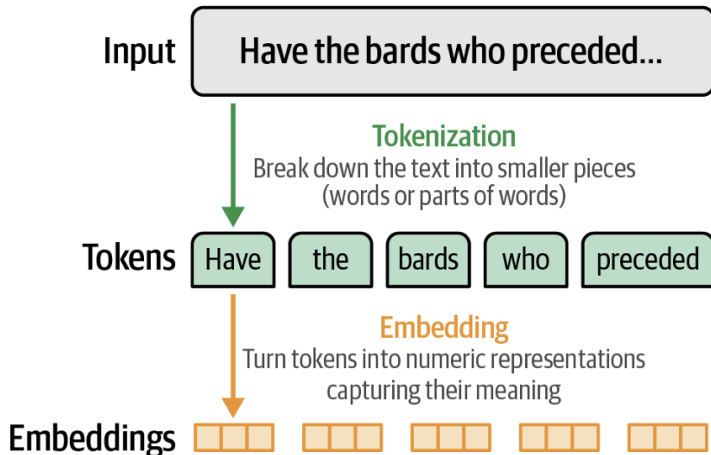
"I love ChatGPT!" $\rightarrow$ ["I", "love", "Chat", "G", "PT", "!"]

Each token has a unique ID that the model uses internally.

Tokenization

Tokenization is the process of converting raw text into small, discrete units called tokens that a language model can understand.

- 1 Tokens are not only the output of a model, they are also the way in which the model sees its inputs.
- 2 A text prompt sent to the model is first broken down into tokens.
- 3 Before the prompt is presented to the language model, it first has to go through a tokenizer that breaks it into pieces.



Input and Output Roles

Encoding: Human text $\rightarrow$ token IDs

"Hello world" $\rightarrow$ [15496, 995]

Decoding: Model output IDs $\rightarrow$ readable text

[15496, 995, 0] $\rightarrow$ "Hello world!"

Tokenizer is the bridge between humans and LLMs.

Input IDs

```
prompt = "Write an email apologizing to Sarah for the tragic gardening mishap.  
Explain how it happened.<|assistant|>"
```

```
# Tokenize the input prompt
```

```
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to("cuda")
```

```
tensor([[ 1, 14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278, 25305, 293,  
16423, 292, 286, 728, 481, 29889, 12027, 7420, 920, 372, 9559, 29889, 32001]],  
device='cuda:0')
```

Input IDs

GPT-3.5 & GPT-4 GPT-3 (Legacy)

Have the bards who preceded me left any theme unsung?

Clear

Show example

Tokens

13

Characters

53

Have the bards who preceded me left any theme unsung?

Text

Token IDs

BERT Tokenizer

[CLS] english and capital ##ization [UNK] [UNK] show _ token ##s false none
eli ##f = = > = else : two tab ##s : " " three tab ##s : " " 12 . 0 * 50 = 600 [SEP]

The word “capitalization” is encoded as two subtokens: capital ##ization. The ## characters are used to indicate this token is a partial token connected to the token that precedes it.

Token Embeddings

Trained tokenizer

Tokens	
Token ID	Token
0	!
1	"
...	...
50,257	

Language model

Token embeddings	
0	
1	
...	...
50,257	

Tokenizer

```
from transformers import AutoModel, AutoTokenizer

# Load a tokenizer
tokenizer = AutoTokenizer.from_pretrained("microsoft/deberta-base")

# Load a language model
model = AutoModel.from_pretrained("microsoft/deberta-v3-xsmall")

# Tokenize the sentence
tokens = tokenizer('Hello world', return_tensors='pt')

# Process the tokens
output = model(**tokens)[0]
```

```
torch.Size([1, 4, 384])
```

```
for token in tokens['input_ids'][0]:  
    print(tokenizer.decode(token))
```

This prints out:

```
[CLS]  
Hello  
world  
[SEP]
```

```
tensor([[  
  [-3.3060, -0.0507, -0.1098, ..., -0.1704, -0.1618, 0.6932],  
  [ 0.8918, 0.0740, -0.1583, ..., 0.1869, 1.4760, 0.0751],  
  [ 0.0871, 0.6364, -0.3050, ..., 0.4729, -0.1829, 1.0157],  
  [-3.1624, -0.1436, -0.0941, ..., -0.0290, -0.1265, 0.7954]  
]], grad_fn=<NativeLayerNormBackward0>)
```

Text Embedding

While token embeddings are key to how LLMs operate, a number of LLM applications require operating on entire sentences, paragraphs, or even text documents. This has led to special language models that produce text embeddings, a single vector that represents a piece of text longer than just one token. We can think of text embedding models as taking a piece of text and ultimately producing a single vector that represents that text and captures its meaning in some useful form.

Text Embedding

```
from sentence_transformers import SentenceTransformer

# Load model
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

# Convert text to text embeddings
vector = model.encode("Best movie ever!")
```

`vector.shape`

`(768,)`

Text Embedding

